# Tutorial: Admin Getting Started

This tutorial walks you through the admin workflow — from logging in to entering your first exam result. You'll learn the core navigation, understand the role-based dashboard, and complete a real task.

## What you'll need

- Admin credentials (email + password)
- A browser (Chrome, Firefox, or Edge)
- The application URL (e.g., `http://localhost:5173` in development)

## Step 1: Log in

1. Open the application URL.
2. You'll see the login page with the MBK International School branding.
3. Enter your admin email and password.
4. Click **Sign In**.

You'll land on the **Admin Dashboard** — the central hub showing key stats: total students, teachers, classes, and exam progress.

## Step 2: Explore the dashboard

The dashboard displays:

- **Student count** — total enrolled students
- **Teacher count** — active teachers
- **Class count** — classes with assigned subjects
- **Exam progress** — percentage of exams entered this month

The sidebar on the left shows all admin navigation. Each section corresponds to a feature domain.

## Step 3: Set up the academic year

Before entering exams, you need an active academic year and term.

1. Click **Academic** in the sidebar.
2. If no academic year exists, click **Create Year**.
3. Enter the year name (e.g., “2025-2026”), start date, and end date.
4. Toggle **Is Current** to on.
5. Create a term within that year (e.g., “Term 1”).
6. Select the months this term covers.

Now the system knows which term exams belong to.

## Step 4: Assign teachers to classes

Teachers need class-subject assignments before they can enter grades.

1. Click **Class Subjects** in the sidebar.
2. Select a class (e.g., “Grade 5-A”).
3. Select a subject (e.g., “Mathematics”).
4. Assign a teacher.
5. Click **Save**.

Repeat for each class-subject-teacher combination.

## Step 5: Enter an exam result

Now try entering a student's exam result:

1. Click **Exam Reports** or navigate to a class view.
2. Select the class, subject, month, and exam type (e.g., "CA").
3. Find the student in the list.
4. Enter their score (e.g., 85) and total (e.g., 100).
5. Click **Save**.

The exam saves with status "pending" — supervisors verify these before they appear on reports.

## Step 6: Verify it worked

1. Go back to the dashboard.
2. The exam progress counter should update.
3. The student's parent can now see this score in their portal.

## What you built

You've completed the core admin loop: set up academic structure → assign teachers → enter exams → parents see results. Every other admin task builds on this foundation.

## Next steps

- **How to set up academic year and terms** — detailed configuration
- **How to enter and verify exam results** — the full exam workflow

- **How to generate student reports** — monthly, midterm, and final reports

— # How to Set Up Academic Year and Terms

Configure the academic calendar so exams, reports, and attendance are tied to the correct time period.

## Prerequisites

- Admin role
- Access to the Academic section

## Steps

### Create an academic year

1. Click **Academic** in the sidebar.
2. Click **Create Year**.
3. Fill in:
    - **Name:** e.g., “2025-2026”
    - **Start Date:** first day of the academic year
    - **End Date:** last day of the academic year
    - **Is Current:** toggle ON (only one year can be current)
4. Click **Save**.

### Create terms within the year

1. In the academic year view, click **Add Term**.
2. Fill in:
    - **Name:** e.g., “Term 1”, “Term 2”, “Term 3”
    - **Start Date:** first day of the term

- **End Date:** last day of the term
- **Is Current:** toggle ON for the active term
- **Months:** select which calendar months this term covers (e.g., January, February, March)

3. Click **Save**.

## Configure grade scales

1. In the Academic section, find **Grade Scales**.
2. Define score ranges and letter grades:

| Min | Max | Grade | Remark | GPA |
|---|---|---|---|---|
| 90 | 100 | A | Excellent | 4.0 |
| 80 | 89 | B | Very Good | 3.5 |
| 70 | 79 | C | Good | 3.0 |
| 60 | 69 | D | Pass | 2.5 |
| 0 | 59 | F | Fail | 0.0 |

3. Click **Save**.

## Configure report weights

1. Find **Report Config** in the Academic section.
2. Set the weights for final grade calculation:
   - **CA Weight:** e.g., 30% (continuous assessment average)
   - **Midterm Weight:** e.g., 30%
   - **Final Weight:** e.g., 40%
3. The weights must sum to 100.
4. Click **Save**.

## Verification

- The dashboard shows the current academic year and term.
- Teachers can only enter exams for the current term’s months.
- Reports calculate using the configured weights.

## Troubleshooting

**Problem:** Teachers can’t enter exams for a month. **Fix:** Check that the month is included in the current term’s month list.

**Problem:** Reports show wrong grades. **Fix:** Verify the grade scale ranges are correct and non-overlapping.

**Problem:** Two academic years are marked current. **Fix:** Only one year can be current — uncheck the old one first.

— # How to Create Class-Subject Assignments

Assign teachers to specific class-subject combinations so they can enter exam results and manage their classes.

## Prerequisites

- Admin role
- Existing teachers in the system
- Existing classes and subjects

## Steps

1. Click **Class Subjects** in the sidebar.
2. You'll see a table of existing assignments.
3. Click **Add Assignment**.
4. Select:
    - **Class:** e.g., "Grade 5-A"
    - **Subject:** e.g., "Mathematics"
    - **Teacher:** select from the dropdown of teachers with `assignedClasses` and `assignedSubjects` set
5. Click **Save**.

### Bulk assignments

To assign one teacher to multiple classes:

1. Go to **Users** in the sidebar.
2. Find the teacher and click **Edit**.
3. Under **Assigned Classes**, check all classes they teach.
4. Under **Assigned Subjects**, check all subjects they teach.
5. Click **Save**.

Then create class-subject assignments — the teacher will appear in the dropdown for their assigned classes.

### Remove an assignment

1. In **Class Subjects**, find the assignment.
2. Click the delete icon.
3. Confirm removal.

This doesn't delete the teacher or class — it just removes the teaching assignment.

## Verification

- The teacher's dashboard shows only their assigned classes.
- Teachers can only enter exam results for their assigned class-subject pairs.
- Supervisors see the assignment in the monitoring view.

## Troubleshooting

**Problem:** Teacher doesn't appear in the class-subject dropdown. **Fix:** Check that the teacher's `assignedClasses` and `assignedSubjects` include the target class and subject.

**Problem:** Teacher can't see a class on their dashboard. **Fix:** Create a class-subject assignment linking that teacher to the class.

— # How to Enter and Verify Exam Results

The exam workflow has two stages: teachers enter results, then supervisors verify them. This guide covers both.

## Prerequisites

- Teacher or admin role (for entering)
- Supervisor role (for verifying)
- Active academic year and term
- Class-subject assignments configured

## Entering exam results (teacher)

1. Click **Results** in the sidebar.
2. Select from the filters:
    - **Class:** your assigned class

    - **Subject:** your assigned subject
    - **Month:** a month in the current term
    - **Exam Type:** CA, Homework, Classwork, Quiz, or Attendance
3. The student list loads for that class.
4. For each student:
    - Enter **Score** (e.g., 85)
    - Enter **Total** (e.g., 100)
    - Optionally add a **Comment**
5. Click **Save All**.

Results save with status `pending`.

## Exam types

| Type | Purpose | Required? |
|---|---|---|
| CA | Continuous Assessment | Yes (one of CA or Homework+Classwork) |
| Homework | Take-home work | Yes (with Classwork) |
| Classwork | In-class exercises | Yes (with Homework) |
| Quiz | Quick tests | Yes |
| Attendance | Presence tracking | Visible but not required |
| Midterm | Mid-term exam | Optional |
| Final | End-of-term exam | Optional |

## Monthly entry rules

- **Quiz** is always required for every student
- **Coursework** is required — satisfied by either:
    - CA for every student, OR
    - Both Homework and Classwork for every student

- **Attendance** is visible but not required

## Verifying exam results (supervisor)

1. Click **Verifications** in the sidebar.
2. Review pending entries — grouped by teacher, class, subject, and month.
3. For each entry:
    - Check the number of students entered matches enrollment
    - Review score distributions for outliers
    - Verify correct exam type and month
4. Click **Approve** to publish to parent reports.
5. Click **Reject** with a reason if something is wrong.

Approved results appear in parent and student portals immediately.

## Verification

- Teachers see “pending” status on entered results until verified.
- Supervisors see completion percentages in the monitoring view.
- Parents see results only after supervisor approval.

## Troubleshooting

**Problem:** Teacher can’t enter results for a month. **Fix:** Check the month is included in the current term. Only current term months are available.

**Problem:** Results don’t appear on parent reports. **Fix:** A supervisor must approve the results first. Check the verification queue.

**Problem:** Wrong number of students shown. **Fix:** Verify the class-subject assignment

and student enrollment match.

— # How to Create and Manage Quizzes

Build quizzes with multiple choice and direct answer questions, publish them to students, and grade attempts.

## Prerequisites

- Teacher or admin role
- Assigned classes and subjects

## Creating a quiz

1. Click **Quizzes** in the sidebar.
2. Click **Create Quiz**.
3. Fill in:
    - **Title:** e.g., “Chapter 5 Review Quiz”
    - **Class:** select the target class
    - **Subject:** select the subject
    - **Description:** optional context for students
    - **Open Date:** when students can start
    - **Due Date:** deadline for submissions
    - **Time Limit:** optional minutes (e.g., 30)
    - **Question Order:** “created” (fixed) or “randomized”
4. Click **Save as Draft** or **Publish**.

## Adding questions

1. In the quiz editor, click **Add Question**.

2. Choose the question type:

### Multiple choice

- Enter the **Prompt** (the question text)
- Add options (minimum 2):
  - **Label:** A, B, C, D
  - **Text:** the answer text
- Mark the **Correct Answer**
- Set **Points** (default: 1)

### Direct answer

- Enter the **Prompt**
- Set **Correct Answer** (for auto-grading)
- Optionally add a **Rubric** (for manual grading)
- Set **Points**

5. Repeat for each question.
6. Click **Save**.

## Publishing

- **Draft:** only you can see it
- **Active:** students can take it
- **Closed:** submissions closed, results available

Set status to **Active** when ready for students.

## Grading attempts

1. Click **Grade Quizzes** in the sidebar.
2. Select the quiz.
3. For each attempt:
    - Review the student's answers
    - For multiple choice: auto-graded, review flagged items
    - For direct answer: manually grade using the rubric
    - Enter **Points Earned** for each question
4. Click **Save Grade**.

Attempt status changes from `submitted` to `graded`.

## Viewing results

- **Quiz list** shows attempt counts and average scores
- **Grade Quizzes** shows individual student answers
- **Parent portal** shows quiz results for their children

## Verification

- Students see the quiz in their portal during the open date window
- Scores appear on reports after grading
- Time-limited quizzes auto-submit when time expires

## Troubleshooting

**Problem:** Students can't see the quiz. **Fix:** Check the quiz status is "Active" and today is between the open and due dates.

**Problem:** Time limit didn't work. **Fix:** Time limits require the student to have a browser open during the quiz. If they close and reopen, the timer continues from where it left

off.

**Problem:** Can’t grade a direct answer question. **Fix:** The rubric field guides your grading — there’s no auto-grading for direct answer questions. Enter points manually.

— # How to Generate Student Reports

The system generates three types of reports: monthly, midterm, and final. Each aggregates exam data differently.

## Prerequisites

- Exam results entered and approved
- Academic year and terms configured
- Grade scales defined

## Monthly reports

Shows one month of exam data per subject.

1. Click **Monthly Report** (parent) or **Exam Reports** (admin/teacher).
2. Select the **Student** and **Month**.
3. The report shows per subject:
    - Each exam type score (CA, Homework, Classwork, Quiz, Attendance)
    - Average score for the month
    - Class average for comparison

### How monthly average is calculated

```
Monthly Average = Sum of all exam scores / Number of exams
```

Each exam type contributes equally to the monthly average.

# Midterm reports

Aggregates across multiple months in a term.

1. Click **Midterm Report**.
2. Select the **Student** and **Term**.
3. The report shows:
    - Per-subject scores across the term
    - Subject rank (position among classmates)
    - Class average per subject
    - Highest score in class
    - Overall rank among all students

## How midterm score is calculated

```
Midterm Score = Average of monthly averages for the term
```

# Final reports

Combines CA, midterm, and final exam with configurable weights.

1. Click **Final Report**.
2. Select the **Student** and **Term**.
3. The report shows:
    - CA average (weighted)
    - Midterm score (weighted)
    - Final exam score (weighted)
    - Total final score
    - Letter grade (A-F)

- Pass/Fail status
- Teacher comment
- Principal comment

### How final score is calculated

```
Final Score = (CA Average × CA Weight) + (Midterm × Midterm Weight)
```

Default weights: CA 30%, Midterm 30%, Final 40%.

### Grade scale

| Score | Grade | Remark |
|---|---|---|
| 90-100 | A | Excellent |
| 80-89 | B | Very Good |
| 70-79 | C | Good |
| 60-69 | D | Pass |
| 0-59 | F | Fail |

Passing threshold: 60.

## Verification

- Monthly reports update immediately when new exams are approved
- Midterm reports recalculate when any month in the term changes
- Final reports reflect the latest grade scale and weight configuration

## Troubleshooting

**Problem:** Report shows no data. **Fix:** Check that exams exist for the selected student, month/term, and that they're approved (status = "approved").

**Problem:** Wrong grade on final report. **Fix:** Verify the report config weights sum to 100 and the grade scale is correct.

**Problem:** Student rank seems wrong. **Fix:** Rank is calculated across all students in the same class. Verify the student's `className` matches the class being compared.

— # How to Use the AI Lesson Planner

Create lesson plans, get AI-powered feedback, and submit for supervisor review.

## Prerequisites

- Teacher role
- Assigned classes and subjects

## Creating a lesson plan

1. Click **Lesson Plans** in the sidebar.
2. Click **Create Plan**.
3. Fill in:
    - **Title:** e.g., "Week 12 - Fractions and Decimals"
    - **Subject:** select from your assigned subjects
    - **Class:** select from your assigned classes
    - **Week Label:** e.g., "Week 12"
4. Click **Save**.

# Adding periods

The lesson plan covers a school week (Saturday through Wednesday, 5 days).

1. In the plan editor, click a day tab (Saturday, Sunday, etc.).
2. For each period, fill in:
    - **Period Number:** 1-6 (or however many periods your school has)
    - **Topic:** what you're teaching (e.g., "Introduction to Fractions")
    - **Objective:** learning objective (e.g., "Students will identify numerator and denominator")
    - **Activities:** what students will do (e.g., "Worksheet exercises, group discussion")
    - **Slide Number:** optional reference to presentation slides
    - **Is Free:** toggle if the period is free (no teaching)
3. Click **Save Periods**.

## Period details

For each period, you can add structured activities:

1. Click **Add Activity** on a period.
2. Fill in:
    - **Activity:** description
    - **Time:** duration (e.g., "15 min")
    - **Resource:** materials needed
    - **Place:** location (e.g., "Classroom", "Lab")

# Submitting for AI review

1. After filling in all periods, click **Submit for Review**.
2. The system sends your plan to the AI review edge function.
3. Wait for the review (typically 10-30 seconds).

## AI review scores

The AI evaluates your plan across 10 categories:

| Category | What it measures |
|---|---|
| Learning Objectives | Clarity, specificity, measurability |
| Lesson Structure | Flow, timing, transitions |
| Student Engagement | Active learning, participation opportunities |
| Teaching Strategies | Variety, appropriateness |
| Differentiation | Adapting for different learners |
| Assessment Methods | How learning is checked |
| Curriculum Alignment | Matches expected standards |
| Classroom Management | Behavior, time management |
| Resources Materials | Appropriateness, availability |
| Overall Quality | Holistic assessment |

Each category gets a score (0-100) and an explanation.

## Performance levels

| Percentage | Level |
|---|---|
| 90-100 | Excellent |
| 80-89 | Very Good |
| 70-79 | Good |

| Percentage | Level |
| --- | --- |
| 60-69 | Needs Improvement |
| 0-59 | Requires Significant Revision |

## Supervisor review

After AI review, the plan goes to your supervisor:

1. Supervisor sees the AI scores and executive summary.
2. They add their own comments.
3. They approve or request revision.
4. If revision is requested, you edit and resubmit.

## Verification

- Plan status changes: `draft` → `submitted` → `in_review` → `approved` / `rejected`
- AI review scores appear in the plan detail view
- Supervisors see all pending plans in their queue

## Troubleshooting

**Problem:** AI review failed. **Fix:** Check that all periods have a topic and activities filled in. The AI needs content to evaluate.

**Problem:** Can't submit for review. **Fix:** Ensure the plan has at least one period with content. Empty plans can't be reviewed.

**Problem:** Scores seem low. **Fix:** Read the AI's improvement suggestions. Focus on clearer objectives, more varied activities, and better assessment methods.

— # Architecture Overview

Why the system is built this way, and how the pieces fit together.

## The problem

Schools need a centralized system to manage students, teachers, exams, and reports. Existing solutions are either too complex (enterprise SaaS) or too simple (spreadsheets). MBK International School needs something in between: a web app that handles the full academic workflow while staying simple enough for non-technical staff.

## The approach

### Frontend: React + Vite + Tailwind

**Why React:** Component-based UI fits the dashboard pattern — each role has different views, but they share common UI elements (tables, forms, modals). React's composition model makes this natural.

**Why Vite:** Fast dev server, single-file output via `vite-plugin-singlefile`. The single HTML file deployment means the app can be hosted anywhere — even a static file server — without a build pipeline.

**Why Tailwind CSS:** Utility-first CSS eliminates the need for a custom design system. The theming system (Acanthus, Baroque, Aurora) uses CSS custom properties that Tailwind's `@theme` directive can reference.

## Backend: Supabase

**Why Supabase:** It provides PostgreSQL, authentication, row-level security, and edge functions in one platform. For a school app, this means:

- **PostgreSQL:** relational data model fits students → exams → reports perfectly
- **Auth:** email/password login with session management
- **RLS:** teachers can only see their own classes, parents only their children
- **Edge Functions:** the AI lesson plan review runs as a Supabase edge function

## Client-side routing

**Why not a router library:** The app uses a simple `switch` statement on `currentPath` in `App.tsx`. Each role has its own route set. This is intentional — the app is small enough that a full router library would add complexity without benefit.

The routing works like this:

```
User logs in → RoleContext determines role → App.tsx switches on pa
```

## State management

**Why React Query:** Server state (exams, students, reports) is cached and refetched automatically. Local state (UI toggles, form inputs) stays in component state. No global store needed.

```
Server state → TanStack React Query (cache, refetch, background syn
UI state → useState/useContext (RoleContext, ToastContext)
```

# Key design decisions

## Role-based access at the component level

Each role has its own set of pages. The `AppContent` component checks `session.role` and renders the appropriate route set. This means:

- Admin sees all management pages
- Teacher sees class-specific pages
- Parent sees child-specific pages
- Supervisor sees monitoring pages

There's no shared routing table — each role's routes are explicit.

## Exams as the central entity

Everything revolves around exams:

```
Students → take Exams → per Subject → per Month → in Classes
Teachers → enter Exams → for their assigned Classes
Supervisors → verify Exams → before Parents see them
Parents → view Exam results → on Reports
```

This flow determines the entire data model and UI structure.

## Single-file deployment

The `vite-plugin-singlefile` plugin inlines all JavaScript, CSS, and assets into a single HTML file. This means:

- No CDN or asset server needed
- Can be emailed as an attachment
- Works offline after first load
- Simple deployment: just serve the HTML file

## Trade-offs

**What was gained:** - Simple deployment (single HTML file) - Fast development (Vite HMR) - Type safety (TypeScript throughout) - Security (Supabase RLS)

**What was given up:** - Server-side rendering (SEO doesn't matter for a school app) - Code splitting (single file means everything loads at once) - Traditional routing (no URL-based navigation, no browser back button) - Offline-first (requires network for Supabase calls)

## Alternatives considered

**Next.js:** Would add SSR and routing, but the single-file deployment goal conflicts with Next.js's server-side model. The app doesn't need SEO.

**Firebase:** Similar to Supabase but less SQL-friendly. The relational data model (students → exams → reports) benefits from PostgreSQL's JOIN capabilities.

**Custom backend:** More control but more maintenance. Supabase handles auth, database, and edge functions — the three things a school app needs.

— # Data Model Design

Why the database is structured the way it is, and how the entities relate.

## The problem

A school management system needs to track: - Users (admin, teacher, parent, supervisor) - Students and their class enrollments - Exam results across subjects, months, and exam types - Academic calendar (years, terms, months) - Assignments (which teacher teaches which class-subject) - Reports (aggregated exam data)

The challenge: these entities have complex relationships, and the access patterns differ by role.

## The approach

### Core entity relationship

```
profiles (users)
  ├── students (one profile → many students via parentId)
  ├── exams (one teacher → many exams via teacherId)
  ├── class_subjects (one teacher → many assignments)
  └── auth_id → auth.users (Supabase auth)

students
  ├── exams (one student → many exams)
  ├── attendance (one student → many records)
  └── parentId → profiles (link to parent)

class_subjects
  ├── className + subjectId → subjects
  └── teacherId → profiles
```

### Why profiles instead of users

The `profiles` table stores application-level user data. Supabase's `auth.users` handles authentication. They're linked via `auth_id`:

```
auth.users (Supabase)     profiles (Application)
├── id (UUID)             ├── id (TEXT)
├── email                 ├── auth_id → auth.users.id
└── password_hash         ├── name
                          ├── role
                          ├── assignedClasses
                          └── assignedSubjects
```

This separation means: - Auth is handled by Supabase (secure, maintained) - Application data is in our control (flexible schema) - The `auth_id` link allows session restoration

## Why exams are the central entity

The exam table connects everything:

```
exam
├── studentId → students
├── teacherId → profiles
├── subject (text, denormalized)
├── examType (CA, Homework, Classwork, Quiz, Midterm, Final)
├── month (text)
├── status (pending → approved/rejected)
└── termId → terms (optional)
```

The `status` field creates a workflow: teachers enter → supervisors approve → parents see. This two-step process ensures data quality.

## Why class_subjects exists

Teachers don't just teach "Mathematics" — they teach "Mathematics to Grade 5-A". The `class_subjects` table captures this:

```
class_subjects
├── className: "Grade 5-A"
├── subjectId → subjects
└── teacherId → profiles
```

This enables: - Teachers see only their assigned classes - Supervisors monitor specific class-subject combinations - Reports aggregate by class-subject

### RLS POLICIES

Row-Level Security ensures each role sees only what they should:

```
Teacher: WHERE teacherId = auth.uid()
  → can only see their own exam entries

Parent:  WHERE parentId = auth.uid()
  → can only see their children's exams

Student: WHERE studentId = auth.uid()
  → can only see their own exams
```

Supervisors bypass RLS — they need to see all data for verification.

## TRADE-OFFS

**Denormalized subject on exams:** The `exams` table stores `subject` as text, not a foreign key. This duplicates data but simplifies queries — no JOIN needed for basic exam listing.

**Text IDs:** The app uses TEXT for primary keys (nanoid or similar). UUIDs would be more standard but TEXT is simpler and works with Supabase.

**Term linkage is optional:** Exams can exist without a `termId`. This allows flexibility but means reports must handle orphaned exams.

## Alternatives considered

**Normalized subject:** Using `subjectId` as a foreign key on exams would prevent typos but add a JOIN to every exam query. The denormalized approach was chosen for query simplicity.

**Separate exam_types table:** An enum or lookup table for exam types would be more normalized but the types are fixed and small — an enum in the CHECK constraint is sufficient.

**Soft deletes:** Adding a `deletedAt` timestamp would allow recovery but adds complexity. The current approach uses hard deletes with CASCADE.

—